

Group Topic 2-Page Report

jg2399
May 2026

<https://github.com/j2399/LoRA-reimp/tree/main>

Introduction

Models these days are incredibly large, making it infeasible to store multiple fine-tuned versions for separate tasks. I re-implement *LoRA: Low-Rank Adaptation of Large Language Models* [1], by Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. The paper's key insight is that the weight updates needed for task adaptation have low intrinsic rank, meaning you can freeze the pretrained weight matrix and instead train two small low-rank matrices, drastically reducing trainable parameters while maintaining high accuracy.

Chosen Result I aimed to reproduce their accuracy of 96.2 ± 0.5 on the SST-2 dataset, reported in Table 2 of Hu et al. [1], which is within 0.5 of the full fine-tuned model showing LoRA achieves full accuracy with far fewer (about 0.2%) parameters. I chose this result since it demonstrates LoRA's effectiveness clearly and the dataset is straightforward to work with.

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$ 0	94.2 $\pm$ 1	88.5 $\pm$ 1.1	60.8 $\pm$ 4	93.1 $\pm$ 1	90.2 $\pm$ 0	71.5 $\pm$ 2.7	89.7 $\pm$ 3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$ 1	94.7 $\pm$ 3	88.4 $\pm$ 1	62.6 $\pm$ 9	93.0 $\pm$ 2	90.6 $\pm$ 0	75.9 $\pm$ 2.2	90.3 $\pm$ 1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$ 3	95.1$\pm$2	89.7 $\pm$ 7	63.4 $\pm$ 1.2	93.3$\pm$3	90.8 $\pm$ 1	86.6$\pm$7	91.5$\pm$2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.9$\pm$1.2	68.2$\pm$1.9	94.9$\pm$3	91.6 $\pm$ 1	87.4$\pm$2.5	92.6$\pm$2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$ 3	96.1 $\pm$ 3	90.2 $\pm$ 7	68.3$\pm$1.0	94.8$\pm$2	91.9$\pm$1	83.8 $\pm$ 2.9	92.1 $\pm$ 7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5$\pm$3	96.6$\pm$2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8$\pm$3	91.7 $\pm$ 2	80.1 $\pm$ 2.9	91.9 $\pm$ 4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$ 5	96.2 $\pm$ 3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$ 2	92.1 $\pm$ 1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$ 3	96.3 $\pm$ 5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$ 2	91.5 $\pm$ 1	72.9 $\pm$ 2.9	91.5 $\pm$ 5	86.4
RoB _{large} (LoRA)†	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.2$\pm$1.0	68.2 $\pm$ 1.9	94.8$\pm$3	91.6 $\pm$ 2	85.2$\pm$1.1	92.3$\pm$5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9$\pm$2	96.9 $\pm$ 2	92.6$\pm$6	72.4$\pm$1.1	96.0$\pm$1	92.9$\pm$1	94.9$\pm$4	93.0$\pm$2	91.3

Figure 1: Reference result from the LoRA paper showing SST-2 accuracy.

Methodology I finetune RoBERTa-large on the SST-2 sentiment classification task from the GLUE benchmark. I train for 6 epochs with batch size 32, learning rate 1×10^{-4} , AdamW optimizer [7] with weight decay 0.01, and a cosine schedule with 10% warmup. Gradient clipping is applied at norm 1.0. I evaluate using classification accuracy on the SST-2 validation set. I inject LoRA adapters into the query and key matrices of every layer. I then evaluate the results of the finetuned model and compare it to the accuracy of the papers full fine tuned and LoRA model. The method reduces the parameters by freezing the pretrained weight matrix $W_0 \in \mathbb{R}^{d \times k}$ and instead fine-tuning two low-rank matrices $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$, where $r \ll \min(d, k)$. The resulting weight update is:

$$\Delta W = BA.$$

I also added my own modification of a third matrix of size $C \in \mathbb{R}^{r \times r}$. My thought process behind this is to put it between the other two matrices so it does some extra thinking in the low dimension so very few parameters are added. My thought process behind why this would add accuracy is since the pretrained matrix has low rank this low rank embedding contains the important information so extra computation here is worth it.

Results & Analysis

The re-implementation achieved its top validation accuracy of 95.87% on SST-2 using both LoRA with $r = 8$ and LoRA3 with $r = 8$, shown in Table 1. I also achieved 95.64% with LoRA and rank 4, and 95.41% with the additional matrix and rank 4. My results are within the 0.5 error range of the paper’s reported 96.2 ± 0.5 , but are on the lower side. I believe this is due to running on only 50% of the available data for 6 epochs rather than the full dataset. Early runs with 3 epochs and $lr = 3 \times 10^{-4}$ showed $r = 8$ underperforming $r = 4$, which resolved after increasing epochs and lowering the learning rate, suggesting the issue was optimization speed rather than overfitting. The LoRA3 modification performed comparably to standard LoRA — slightly worse at $r = 4$ and equal at $r = 8$. In my seeded run the additional matrix hurt performance slightly, whereas a previous unseeded run showed it increasing performance, suggesting the effect requires further testing to determine whether it consistently boosts accuracy. These results confirm the paper’s core claim. Namely, training less than 0.25% of RoBERTa-large’s parameters is sufficient to recover near full fine-tuning accuracy on SST-2.

Model	Type	Rank	Trainable Params	Val Acc
LoRA (A·B)	Re-implementation	4	395,266 (0.111%)	95.64%
LoRA (A·B)	Re-implementation	8	788,482 (0.222%)	95.87%
LoRA3 (A·C·B)	Additional matrix	4	396,034 (0.111%)	95.41%
LoRA3 (A·C·B)	Additional matrix	8	791,554 (0.222%)	95.87%

Table 1: Results on SST-2 validation set (50% data, 6 epochs, seed 42).

Reflections

I learned that the LoRA adapters really do reduce parameters while maintaining high accuracy. I also learned that adding a third matrix can improve performance even more and that there is room for more research in this area. Some potential future areas for research would be why is the rank of the pretrained matrix so low. As without this lora would not work. Another area of interest would be what matrices or ever other methods to use for what we finetune instead of the pretrained matrix. For example, a systematic way of what to put in place of the matrices A, B, and C. Another idea would be to finetune some other novel method of training instead of matrices.

References

References

- [1] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022. [arXiv:2106.09685](#)
- [2] Liu, Y., Ott, M., Goyal, N., et al. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. [arXiv:1907.11692](#), 2019.
- [3] Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., and Bowman, S.R. *GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding*. EMNLP BlackboxNLP Workshop, 2018.
- [4] Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C.D., Ng, A., and Potts, C. *Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank*. EMNLP, pp. 1631–1642, 2013.
- [5] Wolf, T., Debut, L., Sanh, V., et al. *Transformers: State-of-the-Art Natural Language Processing*. EMNLP System Demonstrations, pp. 38–45, 2020.
- [6] Paszke, A., Gross, S., Massa, F., et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS, 2019.
- [7] Loshchilov, I. and Hutter, F. *Decoupled Weight Decay Regularization*. ICLR, 2019. [arXiv:1711.05101](#)